

KEIKEU 技术手册 · 04

给新人的 Rust 简易入门手册

只学 keikeu 的 Tauri 宿主今天真正使用的 Rust。能沿一条请求走完
全程，能读懂失败，能做一处小而安全的改动，就够开工。

KEIKEU CONTRIBUTORS

RUST 1.88

EDITION 2021

2026-08-11

[下载 PDF](#)

先记住三件事

keikeu 的 Rust 是窄桌面宿主，不是第二套业务后端。它管理窗口、五个 Tauri command、原生能力、唯一 Python Sidecar 与 JSONL 边界。

- 01 先读 `main.rs`、`lib.rs`、`commands.rs`、`bridge.rs`；业务规则不在此处。
- 02 `Option` 表示“可能没有”，`Result` 表示“可能失败”，所有权明确谁拿着 `child`、`session` 和字符串。
- 03 单 worker 串行化是协议正确性；写请求丢响应时返回 `commit_unknown`，绝不自动重试。

权威边界： 本页是给人的非规范说明。运行时事实以 `frontend/src-tauri/` 与通过的测试为准；产品与工程边界最终核对 SPEC、RULES 和 PROJECT。若手册与它们冲突，手册错。

阅读路线

- | | |
|--------------------------|----------------------|
| 01 Rust 只管桌面宿主 | 02 四个文件，把壳拼起来 |
| 03 先认这些 Rust 句子 | 04 所有权：谁拿着哪样东西 |
| 05 Option、Result 与 Serde | 06 Tauri command 是窄门 |
| 07 一条队列，一个 Sidecar | 08 安全改动与最小检查 |

Rust 只管桌面宿主

Rust 把 Vue WebView、原生系统能力与 Python Sidecar 接起来；Paper、Markdown、Vault 和迁移规则仍由 Python 负责。



Rust 拿什么，不拿什么

RUST 拿着	RUST 不拿
Tauri 应用生命周期、一个 Sidecar 子进程、单请求队列	Paper 字段、卡页规则、Markdown 解析与序列化
JSONL 请求编号、session、超时与响应匹配	Vault 目录结构、迁移、搜索、Trash 与 Index 业务规则
原生目录选择，以及执行 Python 已验证的 open/reveal	直接读取或写入作者文件，也不替 Vue 管可见草稿状态

这条边界不是“以后再整理”。它是数据安全设计：JavaScript 不获得作者文件权限，Rust 不复制 Python 的业务判断，Python Core 继续是唯一文件规则拥有者。

当前工具坐标

- frontend/src-tauri/Cargo.toml 使用 Rust 1.88、edition 2021。
- 直接依赖只有 Tauri、Serde、serde_json 与 shell/dialog/opener 三个 Tauri 插件。
- 版本均精确锁定；不要为了“学 Rust”运行 cargo update 或引入新 crate。
- 项目没有直接依赖 Tokio；需要的异步运行时由 Tauri 暴露。

最短判断

改动涉及 Markdown、Paper、Vault、搜索或迁移？先离开 Rust，去找 Python owner。只有窗口、Sidecar、IPC 或原生动作才继续看这里。

CHAPTER 02

四个文件，把壳拼起来

不要从 1100 行的 `bridge.rs` 开始硬啃。先读入口和命令，再顺着一次请求进入 worker。

文件	一句话职责	先看哪里
<code>src/main.rs</code>	二进制入口，只调用 library 的 <code>run()</code>	全文，只有三行
<code>src/lib.rs</code>	组合根：插件、托管状态、五个 command、退出清理	<code>run()</code>
<code>src/commands.rs</code>	Vue IPC 与原生能力的窄边界	五个 <code>#[tauri::command]</code>
<code>src/bridge.rs</code>	Sidecar、握手、单队列、JSONL、失败状态与 12 个测试	<code>BridgeHandle</code> ，再看 <code>Worker</code>

以上路径都相对于 `FRONTEND/SRC-TAURI/`。

启动顺序

- 01 `main.rs` 调用 `keikeu_desktop_lib::run()`。
- 02 `lib.rs` 创建 `tauri::Builder`，注册 shell、dialog、opener 插件。
- 03 `setup` 创建一个 `BridgeHandle`，交给 Tauri 的 `manage` 托管。
- 04 `generate_handler!` 一次注册五个 command，Vue 才能按名字调用。
- 05 Tauri build 成功后进入事件循环；退出时取回同一个 `BridgeHandle`，并调用 `shutdown()`。

```
fn main() {
    keikeu_desktop_lib::run();
}
```

`build.rs` 只是 Cargo 构建脚本，调用 `tauri_build::build()` 生成 Tauri 需要的构建信息；它不是应用启动后的第五个业务模块。

推荐阅读顺序

第一次读时走 `main.rs → lib.rs → commands.rs → frontend/src/bridge.js → bridge.rs`。这样先知道谁调用谁，再看并发细节。倒过来读只会在 trait、channel 和状态机里迷路。

CHAPTER 03

先认这些 Rust 句子

项目里的主要句型是：定义数据、给数据加方法、按状态分支、把错误向上传。先会读，不必先背完整语法书。

绑定、类型与返回值

```
const PROTOCOL_VERSION: u64 = 2;

let id = self.next_id;           // 不可重新赋值
let mut bytes = ...;            // 后面会 push, 所以加 mut

fn validate_response(...) -> Result<(), ()> {
    ...
}
```

- `let` 创建绑定；默认不可重新赋值。确实需要变化才加 `mut`。
- 冒号后是类型；`u64` 是 64 位无符号整数。
- `->` 后是返回类型；`()` 是“不返回有用值”。
- 函数最后一个没有分号的表达式就是返回值。

`struct`、`enum` 与 `impl`

`struct` 把固定字段装成一个类型，例如 `BridgeError` 有 `code`、`message`、`recovery`、`layer`。`impl BridgeError` 再给它增加 `host()`、`protocol()` 等构造方法。

`enum` 表示“只可能是这些形状之一”。`RuntimeStatus` 可以是 Starting、带版本数据的 Ready、带错误的 Blocked，或 Stopped。它比四个互相矛盾的布尔值更难写错。

```
match current_status(status) {
    RuntimeStatus::Blocked { error } => error,
    RuntimeStatus::Stopped => BridgeError::unavailable("Sidecar 已停止。"),
    RuntimeStatus::Starting => BridgeError::unavailable("Sidecar 尚未完成握手。"),
    RuntimeStatus::Ready { .. } => BridgeError::unavailable("Sidecar 状态发生竞争。"),
}
```

`match` 必须覆盖全部可能形状。`{ error }` 顺手取出字段，`{ .. }` 表示其余字段此处不关心。项目也使用 `if let`、`let ... else` 与 `matches!` 处理只关心一两个分支的情况。

路径、可见性、属性与宏

```
use crate::bridge::BridgeHandle;
```

从当前 crate 的 `bridge` 模块引入名字。双冒号 `::` 是路径分隔。

`pub` 与 `pub(crate)`

前者可给 crate 外部使用；后者只给当前 crate 使用。不写则默认私有。

```
#[derive(Clone, Serialize)]
```

让编译器或库生成常规实现，避免手写样板。

`json!({...})` 与 `matches!(...)`

末尾叹号表示宏。宏接受的形状可以比普通函数更灵活。

先跳过

测试中的 `AtomicUsize`、`Ordering::SeqCst` 和 trait object 细节不是第一天必修。先知道它们让 fake child 能安全计数和替换真实进程即可。

CHAPTER 04

所有权：谁拿着哪样东西

所有权不是考试题。它回答一个实际问题：字符串、子进程和状态现在归谁，谁能改，什么时候释放。

`String` 与 `&str`

写法	白话	本项目用途
<code>String</code>	自己拥有一段可增长的 UTF-8 文字	跨 command、channel、thread 或 <code>move</code> closure 传递 method、params、路径
<code>&str</code>	临时借看别人的字符串切片	worker 内部同步函数只读 method/action，不拿走所有权
<code>&self</code>	借用对象，只承诺不直接改它	<code>BridgeHandle::status()</code> ；内部 Mutex 仍可受控更新
<code>&mut self</code>	独占借用对象，可以改字 段	<code>Worker::restart()</code> 、 <code>request()</code> 、 <code>stop()</code>

`move || { ... }` 把 closure 用到的拥有值搬进去。worker thread 可能比创建它的函数活得久，所以不能借用函数栈上马上消失的数据。

Clone 不等于再造一套系统

`BridgeHandle` 可以 clone，但其字段是 channel sender、`Arc<Mutex<...>>` 和共享 child。clone 出来的 handle 仍指向同一个 worker、同一个状态和同一个 Sidecar，不会复制 Python 进程。

```
let jobs = self.jobs.clone();
let fallback = self.clone();
tauri::async_runtime::spawn_blocking(move || {
    // jobs 与 fallback 已被搬进 blocking task
})
```

Option::take() 让 child 只死一次

`TauriChild` 内部存 `Option<CommandChild>`。kill 时用 `take()` 把 child 从 Some 变成 None，再消费 child。以后再次 kill 只看到 None，不会重复使用已经被移动的进程句柄。

Arc<Mutex<T>> 是共享柜子

- `Arc`：多个线程各拿一把指向同一对象的钥匙。
- `Mutex`：一次只允许一个持有者打开柜子。
- `lock()` 可能失败，所以生产代码把失败转为结构化错误或 Blocked，不直接 `unwrap()`。

`SharedChild::kill()` 先在锁内 `take()`，然后释放锁，再执行较慢的 kill。不要把进程操作搬回锁内，否则其他线程可能无谓地一直等柜门。

trait 与 Box<dyn Trait>

`ChildControl` 规定 child 至少会 write 与 kill，`SidecarSpawner` 规定能 spawn。生产环境使用 Tauri 实现，测试使用 Fake 实现；这里已有两个真实用途，所以这不是“为未来”造的抽象。

Option、Result 与 Serde

Rust 不靠异常把失败藏到远处：可能缺失用 Option，可能失败用 Result，跨层数据用 Serde 明确变成 JSON。

Option<T>：有或没有

目录选择器取消时没有路径；worker 启动前没有 session；child 被杀后没有 child。这些都不是神秘空指针，而是必须处理的 `Some(value)` 或 `None`。

```
pub async fn choose_vault_directory(...) -> Result<Option<String>, BridgeError>
```

这句同时表达三种结果：选到目录是 `Ok(Some(path))`，用户取消是 `Ok(None)`，选择器或路径转换失败是 `Err(error)`。

实现末尾的 `.transpose()` 把 `Option<Result<String, E>>` 翻成 `Result<Option<String>, E>`。别把取消误报成错误。

Result<T, E>：成功或失败

?

遇到 Err 就立刻向调用者返回；遇到 Ok 则取出里面的值继续。

`map_err(...)`

把底层库错误翻译成稳定的 `BridgeError`，不把插件内部细节漏给界面。

`ok_or_else(...)`

把缺失的 Option 变成带上上下文的错误，并且只在确实缺失时构造错误。

`unwrap_or_else(...)`

失败时使用保守 fallback，例如返回当前 runtime status。

错误也要有稳定形状

```
pub struct BridgeError {
    pub code: String,
    pub message: String,
    pub recovery: String,
    pub layer: String,
}
```

`code` 给程序判断，`message` 给人看，`recovery` 指明恢复入口，`layer` 说明错误归属。不要把它降级成一段随意字符串。

Serde 把 Rust 类型送过边界

- `#[derive(Serialize, Deserialize)]` 生成 JSON 转换能力。
- `RuntimeStatus` 使用 `#[serde(tag = "state", rename_all = "snake_case")]`，Vue 因此看到 `state: "ready"`。
- `serde_json::Value` 让 host 只搬运协议 envelope，不在 Rust 复制全部 Python DTO。
- `json!({...})` 组装 hello 与业务请求；发送前用 `to_vec` 编码并补一个换行。

什么时候可以 panic

正常失败返回 `Result`。当前生产代码的 `expect("failed to build keikeu desktop host")` 位于应用根本无法创建的启动点；`unreachable!` 位于 action 已由前一步验证的分支。测试里的 `unwrap()` 用于让测试立即暴露“不该失败”的情况。不要把测试习惯复制进运行时 I/O。

CHAPTER 06

Tauri command 是窄门

Vue 不能随意碰文件或子进程。它只能按名字穿过五个已注册 command，每个入口都把权限限制在最小范围。

COMMAND	用途	是否进 PYTHON
<code>runtime_status</code>	读取 Starting / Ready / Blocked / Stopped	否
<code>bridge_request</code>	承载获准的业务 protocol method	是
<code>restart_sidecar</code>	杀旧 child、启动新 child、重新 hello	只做 hello
<code>choose_vault_directory</code>	打开原生文件夹选择器，返回路径或取消	否
<code>open_system_target</code>	打开或在 Finder 显示已验证目标	先验证，再执行

从 JavaScript 到 Rust

```
// frontend/src/bridge.js
invoke("open_system_target", { action, relativeTarget })

// frontend/src-tauri/src/commands.rs
pub async fn open_system_target(
    action: String,
    relative_target: String,
    ...
) -> Result<(), BridgeError>
```

Tauri 默认把 JavaScript 的 camelCase 参数映射到 Rust 的 snake_case，所以 `relativeTarget` 对应 `relative_target`。改名字时必须同时看两边。

State<'_, BridgeHandle> 是同一个托管句柄

`lib.rs` 在 `setup` 中调用 `app.manage(...)`；command 用 `State<'_, BridgeHandle>` 借到它。`'_` 表示借用生命周期由编译器推断，新人此处不需要自己发明生命周期名字。

为什么目录与 open/reveal 分成两条路

目录选择是纯原生意图：Rust 打开 picker，取消返回 None；若系统路径不能无损表示为 UTF-8，就返回错误。不要用 `to_string_lossy()` 静默改路径。

open/reveal 涉及作者文件，必须先走内部 `system.resolve_target` 让 Python 校验 action、Vault locator 与相对路径。Rust 只把 Python 返回的已验证绝对路径交给 opener。公开 `bridge_request` 不允许 Vue 直接调用这个内部 method。

async 不会自动消灭阻塞

标准库 channel 的 `recv()`、blocking picker 与系统 opener 都会阻塞线程，所以 command 用 `tauri::async_runtime::spawn_blocking` 把它们放到专门的 blocking executor。不要在 async command 里直接等阻塞 I/O。

权限边界

前端 capability 没有通用 shell/opener 权限。不要为了省一层 command 把它们开放给 Vue；那会绕过 Python 的路径验证与当前审计边界。

CHAPTER 07

一条队列， 一个 Sidecar

并发调用者可以很多，但真正与 stdout 对话的 worker 只有一个。它一次发送一个请求并等回对应响应。

```
多个 Tauri command
  ↓ WorkerJob (mpsc::Sender 可 clone)
唯一 keikeu-sidecar-worker 线程
  ↓ 一次写一行 JSON, 等待同 id 响应
唯一 Python Sidecar
  ↓ stdout event
Tauri async task → std::sync::mpsc → worker
```

三个并发零件

`mpsc::channel()`

多个 producer 把 job 发给一个 consumer; consumer 就是唯一 worker。

`mpsc::sync_channel(1)`

每个 job 自带一条容量为 1 的回复通道, 调用者只等自己的结果。

`Arc<Mutex<...>>`

让 Tauri 线程、worker 与 shutdown 路径共享 runtime status 和当前 child。

worker 在 `run()` 的循环里逐个接收 Request、ResolveTarget、Restart、Shutdown。即使两个前端调用同时到达, 写入顺序和 request id 仍由一个 worker 决定。这里的串行化不是暂时偷懒, 而是避免两次 stdout 响应串线。

启动与重启都先握手

01 杀掉旧 child, 清空 events 与 session, 把 next id 重置为 1。

02 启动打包的 `keikeu-sidecar`。

03 发送 protocol v2 的 `system.hello`, hello 不带旧 session。

04 检查响应 v、id、protocol version、非空 session 和版本字段。

05 成功后设为 Ready; 失败则杀 child 并进入 Blocked。

失败语义比“再试一次”重要

情况	RUST 结果	为什么
hello 版本不兼容	<code>protocol_mismatch</code> + Blocked	错配 host/sidecar 不能继续猜
只读请求超时、EOF 或 child 崩溃	<code>sidecar_unavailable</code> + Blocked	没有磁盘 mutation 需要对账

情况	RUST 结果	为什么
只读响应 JSON、v、id 或 envelope 无效	<code>protocol_mismatch</code> + Blocked	收到数据，但不能信任协议
写请求发送后响应丢失或无效	<code>commit_unknown</code> + Blocked	磁盘可能已提交，自动重发会重复 mutation

`commit_unknown` 不是普通网络错误。界面必须保留 durable intent，重启 Sidecar 后走对应只读刷新或 `paper.reconcile_save`，不能再发原 mutation。

stdout 是协议，不是日志板

Rust 把每条 Sidecar stdout event 当作等待中的响应。Python 若向 stdout 打一行调试文字，Rust 会把它解析为 JSON，失败后阻塞整个 bridge。Sidecar 日志只能走 stderr；当前 Rust 会忽略 stderr 内容，不把私密作者数据带回前端。

CHAPTER 08

安全改动与最小检查

最好的第一处 Rust 改动不是新架构，而是一处边界明确、测试能钉住的小改动。先找 owner 和 caller，再写最小 diff。

动手前的五步

```
git status --short --branch
git branch --show-current
```

- 01 先跑上面两条 Git gate 命令，确认当前分支和工作树状态。
- 02 用 `rg` 找准备改的函数、所有 caller，以及同一 command 在 Vue 和 Python 的对应点。
- 03 判断责任：是 Tauri/Sidecar/IPC/原生动作，还是应该留在 Python 或 Vue。
- 04 若新增 protocol method，先明确 readonly 或 mutation；这会改变 timeout 与失败恢复。
- 05 做最小 patch，跑格式、编译和 12 个 Rust 单元测试，再决定是否需要全栈 smoke。

新增一个 bridge method 不是改一个字符串

至少要同步检查四处：Python method 分类与 dispatcher、Rust `public_policy`、Vue 调用者、两边测试。内部 `system.hello` 与 `system.resolve_target` 仍不能从公开 `bridge_request` 直达。

最小命令集

```
cargo fmt --manifest-path frontend/src-tauri/Cargo.toml --check
cargo check --manifest-path frontend/src-tauri/Cargo.toml
cargo test --manifest-path frontend/src-tauri/Cargo.toml
```

完整桌面联调还要准备 Python Sidecar 和 Vue:

```
.venv/bin/python scripts/build_sidecar.py
npm --prefix frontend run tauri:dev
```

不要把 `cargo run` 当成 keikeu 的完整启动方式; 它不能替代 Vue 构建和打包 Sidecar。

现有 12 个测试守住什么

- 每个公开 method 的 readonly/mutation 分类; 内部 method 不可公开直达。
- hello 协议版本、session 与响应 id 匹配。
- 并发 caller 被串行化, id 单调递增。
- validated system target 只留在 Rust 内部。
- mutation 的 timeout、EOF、错 id、坏 JSON 与坏 envelope 都只产生一次 `commit_unknown`。
- 只读 response loss、协议损坏与 child crash 使用不同错误。
- restart 创建新 session 并杀旧 child; shutdown 杀 child 并标记 Stopped。

交付前扫一遍

- 没有把业务规则、Markdown 或文件写入塞进 Rust。
- 没有给 Vue 新增通用 shell/opener 权限。
- 没有自动重试 mutation, 也没有把 `commit_unknown` 改成普通失败。
- 没有在生产 I/O 路径增加 `unwrap()`。
- 没有 async command 里直接执行阻塞等待。
- 没有用 `to_string_lossy()` 静默改变路径。
- 没有新增 crate; 若确实需要, 先说明现有代码和 stdlib 为什么不够。
- 报告实际跑过与未跑的检查, 不把 Cargo test 冒充 Tauri 应用 smoke。

第一个练习

先从 `frontend/src/bridge.js` 的 `getRuntimeStatus()` 追到 `commands::runtime_status`, 再追到 `BridgeHandle::status()`。这条链没有 Python 和磁盘 mutation, 最适合练习模块、State、借用、Mutex 与 Serde。

来源与继续阅读

1. 项目源码: `main.rs`、`lib.rs`、`commands.rs`、`bridge.rs`。
2. 构建坐标: `Cargo.toml`、`tauri.conf.json`、`capabilities/main.json`。
3. 相邻边界: `frontend/src/bridge.js`、`sidecar.py`、`protocol.py`。
4. The Rust Programming Language: 所有权与借用。
5. Rust Book: 枚举、Option 与模式匹配; 错误处理。
6. Rust Book: 并发; 标准库 mpsc、Arc、Mutex。
7. Tauri: 从前端调用 Rust、嵌入外部 Sidecar、Tauri async runtime API。

检查时间: 2026-08-11。现场为 `rustc 1.88.0`、`cargo 1.88.0`; `cargo fmt --check`、`cargo check` 与 `cargo test` 均通过, Rust 单元测试为 `12 passed`。这些是工程检查, 不等于 Tauri GUI smoke、平台兼容或产品验收。